

Continuous Compliance Monitoring of Water Treatment Plants Using a Fog-to-Cloud Architecture

Rakesh Kunchala
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
Student ID: X25176862
x25176862@student.ncirl.ie

Abstract—A water treatment plant must hold several quality parameters inside regulated bands continuously, yet an operator who learns of a breach only from a periodic grab sample may act hours after the water has already left the plant. This paper presents a working fog-to-cloud pipeline that watches two treatment plants without interruption. Each plant carries five instruments, turbidity, pH, free chlorine, flow rate, and line pressure, that are sampled locally and dispatched to a fog gateway at the plant. The gateway folds each stream into a fixed time window, reduces it to summary statistics, and evaluates the window against compliance rules that encode recognised drinking-water limits, so a breach is flagged in the same cycle it occurs rather than after a laboratory turnaround. Compact per-window aggregates cross a managed message queue into an event-driven ingestion function that persists them to a key-value store; a second, independently deployed function serves a dashboard programming interface behind a managed gateway, and the operator page is delivered as static content from object storage. The system is built in Node.js and provisioned to a real Amazon Web Services account by a single infrastructure-as-code apply that created twenty-four resources. Measurement against the live endpoint shows the pipeline healthy end to end within about a minute of start-up, the stored record count rising across successive polls, and both plants rendering live, compliance-annotated readings in a browser. A defect that surfaced only in the deployed function, an argument-shape collision on the serverless runtime, is documented together with its fix.

Index Terms—fog computing, edge computing, water quality monitoring, regulatory compliance, serverless, message queue, Internet of Things.

I. INTRODUCTION

Treated drinking water is a regulated product. A plant is expected to keep turbidity low, keep a disinfectant residual present throughout the distribution network, hold pH within a narrow band, and maintain line pressure, and it is accountable for doing so continuously rather than on average. The World Health Organization's drinking-water guidelines set out the parameters that matter and the reasoning behind their limits: turbidity must be low both because it is an aesthetic and operational concern and because particulate matter shields microorganisms from disinfection; a free chlorine residual must be maintained through the network to guard against recontamination; and pH must be controlled because disinfection efficiency and pipe corrosion both depend on it [1]. The common thread is that these are not one-off acceptance tests but conditions that must hold at all times.

Traditional monitoring does not match that requirement well. A grab sample sent to a laboratory characterises the water at one instant and reports back after a delay, so a transient excursion, a chlorine dip after a dosing fault, a turbidity spike after a filter breakthrough, can pass entirely between samples, or be discovered only once the affected water has already been distributed. Continuous instrumentation closes that gap, and a body of work has shown that networked sensors can monitor water quality in real time and at acceptable cost [2], [3]. The open question for a systems designer is not whether to instrument the plant but where the resulting data should be processed and how it should travel to the people who act on it.

Streaming every raw reading to a central cloud is the wrong default. A plant with several fast instruments produces readings faster than any of them is individually useful, the uplink from a remote plant is finite and occasionally unreliable, and a monitoring view that goes dark whenever connectivity dips is of least use during exactly the upsets that stress the plant. Fog and edge computing offer a better division of labour by moving computation toward the data source [4]. Fog computing in particular inserts a compute tier between the instruments and the cloud, so that windowing, summarisation, and time-critical rule evaluation happen at the plant, while the cloud keeps the roles it performs best: durable storage, elastic query serving, and a globally reachable interface [5], [6]. The cloud services used here follow the standard on-demand, elastic, measured model of cloud computing [7].

This paper reports the design, implementation, and live deployment of such a pipeline for water treatment monitoring. The deployment watches two plants, each carrying five instruments. The objectives are fourfold. First, to reduce raw readings to compact per-window aggregates at a fog tier at the plant, so that only summaries cross the link to the cloud. Second, to evaluate compliance rules that encode recognised drinking-water limits at the edge, so that a breach is flagged in the cycle it is observed. Third, to build a cloud back end that ingests aggregates reliably, stores them durably, and serves them to an operator dashboard without the write path and the read path contending. Fourth, to deploy the whole system to a real cloud account and verify it end to end against the live endpoint. The remainder of the paper describes the architecture

and its design decisions, the implementation and its automated tests, the deployment including a defect that appeared only once deployed, an evaluation drawn from the running system, and a concluding reflection.

II. SYSTEM ARCHITECTURE AND DESIGN

The system is a linear ingestion pipeline with a separate read path for the operator view. Instruments produce readings; a fog gateway at the plant windows, summarises, and screens them; a message queue carries aggregates to the cloud; an event-driven function persists them; and an independent function reads the stored data back out to a dashboard. Fig. 1 shows the deployed topology. This section takes each tier in turn and then weighs the main alternatives.

A. Edge Sensing and Fog Aggregation

Each plant is represented by five independent sensor processes, one per measured quantity: turbidity in nephelometric turbidity units, pH, free chlorine in parts per million, flow rate in litres per second, and line pressure in bar. A sensor generates values with a bounded random walk: each new reading steps a small random amount from the previous one and is clamped to a physically plausible range, so a stream drifts realistically rather than jumping arbitrarily, and the hydraulic quantities are given a larger step than the chemistry because they swing faster tick to tick. Every sensor is governed by two independently configurable intervals, one setting how often a value is produced and the other how often the accumulated values are dispatched to the gateway. Separating sampling from transmission lets a fast-changing quantity be sampled often while still being sent to the edge in economical batches, mirroring real deployments in which the two rates are tuned separately. Each dispatch is a small JSON object naming the sensor type, the plant identifier, and the unit, plus the short list of timestamped values accumulated since the previous dispatch, a payload of a few hundred bytes.

The fog gateway is a small Node.js and Express web service. It accepts posted batches and keeps one running accumulator per combination of sensor type and plant. Rather than retain raw readings, it folds each incoming value immediately into a compact record holding the count, running sum, minimum, maximum, and most recent value, so its memory footprint stays flat regardless of how many readings arrive. On a fixed cadence it closes the current window: each accumulator is sealed into a summary carrying the window bounds, the count, the minimum, the maximum, the mean, and the latest value, and that summary is evaluated against the compliance rule for its sensor type before being forwarded.

B. Compliance Rules at the Edge

Four rules are evaluated, one per compliance-relevant stream; flow rate is retained for display and trend analysis but carries no rule, because a high or low instantaneous flow is an operational quantity rather than a quality breach. Table I lists the rules alongside the drinking-water rationale each encodes. Three rules watch the window mean, because

TABLE I
EDGE-EVALUATED COMPLIANCE RULES

Stream	Rule (per window)	Breach raised
Turbidity	mean above 5 NTU	Turbidity alert
Free chlorine	mean below 0.2 ppm	Under-chlorination
pH	mean below 6.5	Acidic violation
Line pressure	minimum below 2 bar	Low-pressure fault
Flow rate	(informational, no rule)	None

a sustained average is a more meaningful signal of a genuine excursion than a single noisy sample; the pressure rule instead watches the window minimum, because the worst reading in the window, not its average, is the signal that distribution integrity may have been lost. Evaluating these rules at the edge means a breach is detected in the same cycle as the readings that reveal it, with no round trip to the cloud, which is precisely the property a laboratory grab sample cannot provide.

When a window closes the gateway does not send one message per plant per stream. It collects every summary produced in that cycle and hands the whole set to a single batched send toward the queue, splitting the set into chunks only where the queue interface caps the number of entries in one batched call. For a two-plant deployment producing up to ten summaries per cycle, this turns ten network calls to the cloud into one, lowering request cost and per-message overhead.

C. Decoupled Cloud Ingestion and Storage

Aggregates leave the fog tier through a managed message queue rather than a direct call into the storage function. The queue is a buffer and a shock absorber: it holds messages durably until they are consumed, so a slow or briefly unavailable consumer neither loses data nor pushes back on the gateway, and a burst of aggregates is smoothed to a rate the consumer can sustain. This is the decoupling that message-oriented middleware provides between producers and consumers [8]. The decoupling covers the consumer side of the queue, not the hop onto it: if the gateway's batched send fails, the window has already been sealed and its accumulators cleared, so the gateway logs the failure and proceeds to the next window rather than retrying or buffering locally. This is a stated trade-off, accepted at the data volume a two-plant deployment produces, where the next window arrives seconds later.

An event-driven ingestion function consumes the queue. It is invoked with a batch of messages, transforms each aggregate into a stored item, and writes it to a managed key-value store. The store uses a two-part key. The partition key is the sensor type, which groups all readings for a given quantity together and makes the dominant query, "give me the recent windows for this stream," a single efficient lookup. The sort key concatenates the window-end timestamp with the plant identifier. The timestamp gives a natural chronological order within each partition, and appending the plant identifier is

Water Treatment Utility | Fog-to-Cloud Deployment Topology

AWS Cloud (us-east-1)

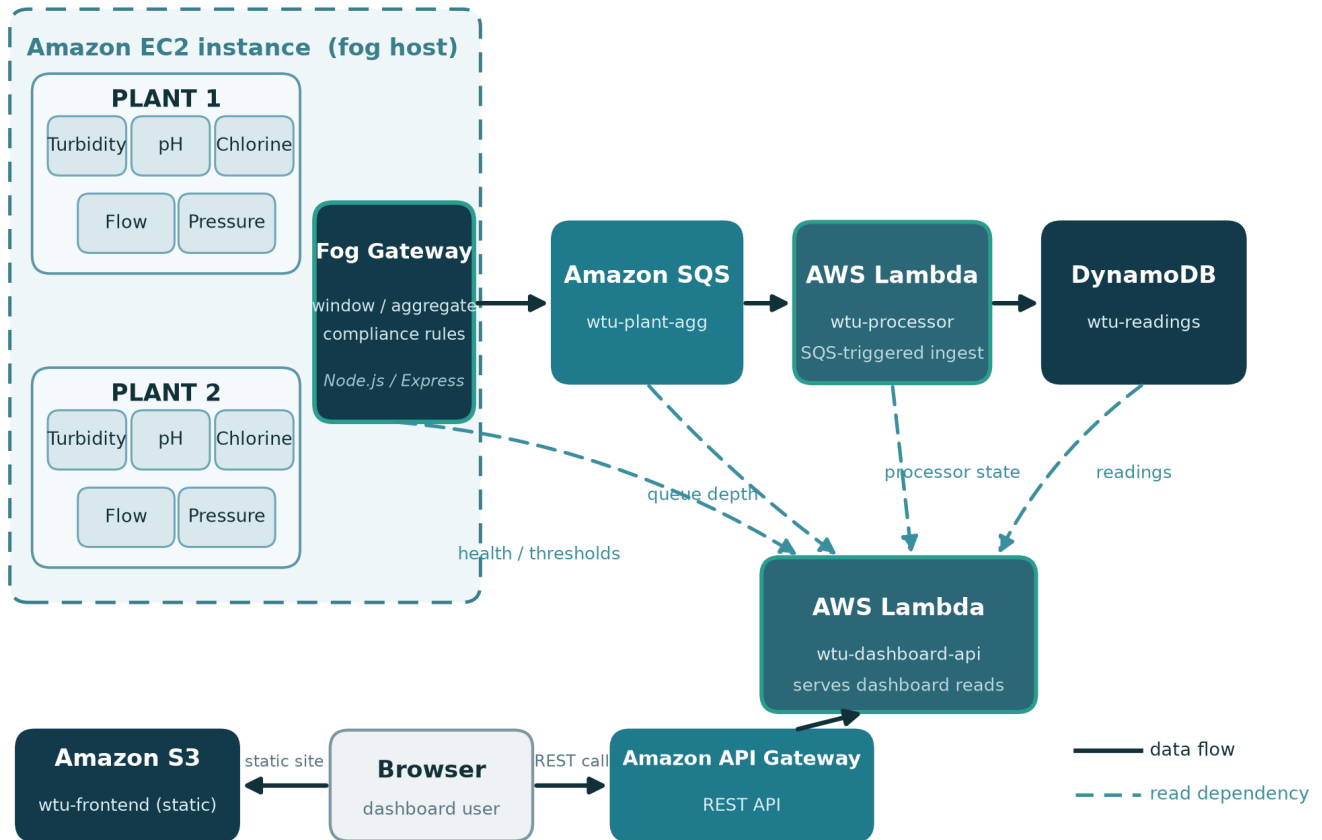


Fig. 1. Fog-to-cloud deployment topology. The fog gateway and the ten sensor processes run in containers on a single edge host at the plant. Solid arrows are the ingestion data flow from instruments through the queue and the ingestion function into the store. Dashed arrows are the read dependencies the dashboard function assembles from the gateway, the queue, the ingestion function, and the store. The browser loads static assets from object storage and calls the dashboard programming interface through the managed gateway.

deliberate: when both plants close a window for the same stream in the same cycle, their two items share a partition key and a window-end time, and without the plant identifier in the sort key the second write would overwrite the first. Concatenating the identifier makes the two keys distinct and lets both plants coexist in one partition without collision.

D. Serverless Dashboard Tier and Frontend

The operator view is served by a second cloud function deployed independently of the ingestion function, so the read path and the write path scale and fail separately. This function answers a small read-only programming interface: a plants endpoint returning the latest reading for every stream on both plants, a per-plant detail endpoint, a readings endpoint returning a recent series for one stream and plant to draw the trend charts, a health endpoint, a back-end statistics endpoint, and a thresholds endpoint that proxies the gateway's published rules. The health endpoint reports four independent booleans,

that the fog gateway is reachable, the queue is reachable, the ingestion function is active, and the pipeline is flowing, meaning the freshest stored window is recent, together with the age of the freshest window. Assembling this view makes the dashboard function read from four sources: the store for readings and record counts, the queue for depth, the ingestion function for its state, and the gateway for its health and thresholds. These are the dashed edges in Fig. 1. Unlike the ingestion function, whose work is uniform, the dashboard function multiplexes several route handlers; each is a small query against one of those sources, dispatched by a compact method-and-path matcher.

The dashboard function is reached through a managed programming-interface gateway that exposes a public REST endpoint. The static frontend, a single page, a stylesheet, the page script, and a locally vendored charting library, is served from object storage. The page renders a plant-readings table with each stream's live value and a meter against its plausible

range, a per-plant compliance badge, and a panel of window-average trend charts comparing the two plants; a header strip carries the four pipeline indicators. The page script re-fetches the plants view, the health endpoint, and the back-end statistics together on a short interval and repaints from each response, so the view stays current without a manual reload. The frontend learns which programming-interface origin to call from a single inline value in the page that is substituted with the real gateway origin at upload time and read once by the page script; in local development, where one process serves both the interface and the page, the value is left unset and calls default to the same origin.

E. Design Rationale: Alternatives Considered

Each of the following choices was made for a stated reason rather than by default.

A queue rather than a direct function call. The gateway could invoke the storage function directly and synchronously, removing the queue and one hop of latency. This was rejected. A synchronous call couples the edge to the availability and speed of the cloud function: if the function is throttled, cold, or briefly unavailable, the gateway blocks or drops data, and back pressure propagates to the instruments. The queue breaks that coupling, absorbs bursts, retries delivery on the consumer's behalf, and lets the gateway consider an aggregate handed off the instant it is enqueued [8]. The slight extra latency is irrelevant for monitoring on a window cadence, whereas the durability and decoupling are exactly what an imperfect plant uplink needs.

A key-value store rather than a relational database. A relational database would offer joins and ad hoc queries. It was set aside for this workload. The access pattern is narrow and known in advance: append time-ordered aggregates, then read back the most recent windows for a stream. A key-value store built for that pattern of high-volume writes and key-based reads gives predictable performance at scale without the operational weight of a relational engine, the trade-off the original Dynamo design articulated and the managed service inherits [9], [10]. The chosen key scheme turns the one query the dashboard needs into a single partition lookup.

Event-driven functions rather than an always-on server. The ingestion and interface tiers run as event-driven functions. This suits a workload that is bursty and, for the interface, intermittent: the account pays per request rather than for idle capacity, and the platform scales the function horizontally as load varies without any capacity being provisioned in advance. The known cost is cold-start latency after a function has been idle [11]; here the effect is minor, because a steady stream of queue messages keeps the ingestion function warm and the dashboard's short polling interval does the same for the interface.

F. Security Posture

The edge host exposes a single inbound port carrying only the gateway's health and threshold endpoints; it has no remote-login key pair and is administered through the cloud

TABLE II
AUTOMATED TEST COVERAGE BY MODULE

Module	Tests
Sensors	11
Fog gateway	51
Ingestion function	10
Dashboard interface	43
Total	115

provider's systems-management channel, removing the most common remote-access attack surface. Both cloud functions run under the account's shared laboratory role, because the account cannot create new roles; the stated production intent is to split this into least-privilege roles, granting the ingestion function only the ability to receive and delete queue messages and write items, and the dashboard function only read-oriented permissions. The dashboard interface is intentionally unauthenticated because it exposes only aggregated, non-personal plant telemetry; because it is served from one origin and called from another, every response, including error responses, carries an explicit cross-origin header so a browser accepts the reply under the rules that govern cross-origin requests [12].

III. IMPLEMENTATION

The entire stack is implemented in Node.js with no build step and no TypeScript. The fog gateway is built on the Express web framework; the cloud clients for the queue, the store, and the functions come from the official provider software development kit for JavaScript. The dashboard's trend charts are drawn with a charting library vendored into the project and served locally rather than from a content delivery network, so the page carries no third-party runtime dependency. For local development and continuous integration the full stack runs under Docker Compose against a local emulator of the cloud services, so the same code runs unchanged against the emulator or the real account by changing only endpoint and credential configuration.

A. Automated Test Suite

Testing uses the test runner built into the Node.js runtime, with no external framework. Cloud-facing code accepts an injected client, so the unit tests exercise it against small hand-written fake clients rather than a live service or even the emulator, keeping the suite fast, deterministic, and free of network dependence. The suite totals 115 tests across the four modules, distributed as shown in Table II. The fog module carries the largest share because it holds the windowing, the statistical reduction, and the compliance rules, the logic most in need of direct coverage.

B. Continuous Integration

A continuous-integration workflow runs on every push and every pull request as two dependent jobs. The first installs dependencies and runs the unit suites for all four Node.js

modules. Only if that job passes does the second run: it brings up the emulator-backed Docker Compose stack, executes an end-to-end check that pushes data through the sensors, the gateway, the queue, the ingestion function, and the store, and then tears the stack down. Gating the integration job on the unit job keeps feedback fast in the common case, where a logic error is caught by the quick unit suite, while still exercising the assembled system before a change is accepted.

C. Infrastructure-as-Code Deployment

The system was deployed live to a real provider laboratory account in the North Virginia region. Provisioning is expressed as infrastructure as code and applied in an isolated workspace so that it plans only its own resources: a single apply created twenty-four resources with no manual command-line steps, the plan reporting twenty-four to add and none to change or destroy. The live resources comprise a key-value table, a message queue, the ingestion function and the dashboard function reached through a REST programming-interface gateway, an edge compute instance whose firewall admits only the single gateway port and which carries no login key pair, a fixed public address attached to that instance, and two object-storage buckets, one serving the dashboard frontend and one used for deployment staging.

D. A Deployment-Only Defect

One defect did not appear in any unit test or in local Docker Compose, and surfaced only once the dashboard function was invoked on the serverless runtime. To keep the dashboard code testable, its request handler accepts an optional third argument carrying pre-constructed cloud clients, which the unit tests supply as fakes; in production the handler is meant to build its own clients instead. The serverless runtime, however, invokes a handler with three arguments of its own, an event, a context, and a completion callback. The handler had treated any truthy third argument as injected clients, so on the real runtime it received the runtime's callback in that position, tried to read a client off it, and failed with an undefined-property error on the first request, even though every local test passed. The fix narrows the condition: the third argument is treated as injected clients only when it actually carries a store client, and otherwise the handler builds its own, so the fake-client path the tests rely on and the self-constructing path production needs are distinguished by the shape of the argument rather than merely its presence. The episode is a reminder that a runtime's own calling convention is part of a function's contract, and that behaviour visible only in the deployed environment must be verified there, not inferred from local tests alone.

The application code and the infrastructure-as-code configuration are documented in the repository at [GitHub repository URL]. The system was then evaluated by measuring the running deployment, not by inspecting code alone; all figures below were read from the live endpoint.

IV. LIVE EVALUATION

A. End-to-End Liveness

Within about a minute of the edge instance finishing container start-up, the health endpoint reported all four indicators true, the gateway reachable, the queue reachable, the ingestion function active, and the pipeline flowing, and the freshest stored window was a few seconds old. A freshest-window age in single-digit seconds means a reading produced at a plant had already traversed the sensor, the gateway, the queue, the ingestion function, and the store and was being read back by the dashboard, all within seconds. This is the property that distinguishes a live pipeline from a set of separately deployed but disconnected components.

B. Throughput and Accumulation

The stored record count was polled repeatedly across the verification window and was observed to climb across successive polls, confirming that the ingestion function was consuming the queue and writing continuously rather than once. The record-count query itself is written to page through the store's continuation cursor and sum the per-page counts, rather than issue a single count-only scan, because a single scan returns only the first page and would silently understate the total once the table grew; paging keeps the reported total correct regardless of table size. Over the same window the plants endpoint returned live per-plant data for both plants, five current readings each with their window minimum, maximum, and mean, and each plant carried a compliance badge computed from the edge rules. All static frontend assets were fetched successfully from object storage, the cross-origin header was present on the interface responses, and the inline interface origin in the page had been correctly substituted with the real gateway address at upload time.

Fig. 2 is a screenshot of the deployed dashboard rendered in a real browser. It shows the plant-readings table with both plants' live values and range meters, the two per-plant compliance badges, the panel of window-average trend charts, and the header indicators reporting the pipeline healthy. The screenshot is evidence that the system is not merely reachable by a command-line probe but renders correct, live, compliance-annotated content to a human operator.

C. Cost

The design keeps standing cost low. Every cloud service in the pipeline except the edge instance is billed per request and, at the volumes this deployment produces, stays within the provider's free tier: the queue, the two functions, the key-value store, the interface gateway, and the object storage all bill on use. The edge instance is the only continuously billed resource, and because the serverless dashboard and interface do not depend on it being up, only fresh readings and the gateway's own health reading do, it can be stopped while idle without taking the dashboard offline. The standing cost of the system when idle is therefore essentially that of one small instance, and even that can be paused.



Water Treatment Utility Monitor

Two treatment plants — turbidity, pH, chlorination, flow & pressure, live

• Gateway

• Queue

• Lambda

• Pipeline

PLANT READINGS

READING	PLANT 1	PLANT 2
Turbidity 0–15 NTU	0.16 NTU 	1.27 NTU
pH Level 5.5–9 pH	6.84 pH 	6.77 pH
Chlorine 0–3 ppm	0.51 ppm 	1.34 ppm
Flow Rate 5–120 L/s	34.8 L/s 	48.9 L/s
Pressure 0.5–8 bar	4.55 bar 	5.09 bar

• plant-1: compliant

• plant-2: compliant

WINDOW-AVERAGE TRENDS

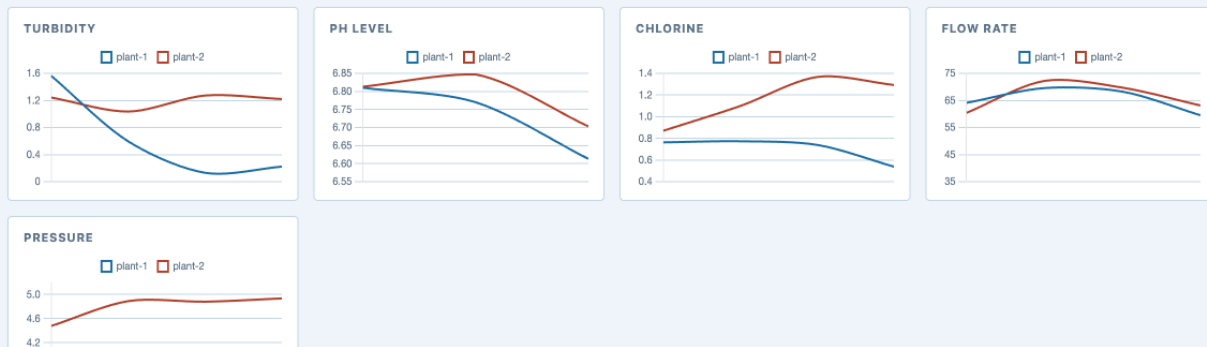


Fig. 2. The deployed operator dashboard rendered live in a browser. The upper table shows each stream’s live value and a meter against its plausible range for both plants, with a per-plant compliance badge; the lower panel compares window-average trends across the two plants; the header carries the four pipeline health indicators.

D. Limitations

Three limitations are stated plainly. First, the sensor and fog tier runs on a single edge instance, a single point of failure for that tier: if the instance stops, new readings stop, though previously stored data and the serverless read path remain available. A production deployment would replicate the fog tier across instances or availability zones. Second, the laboratory account is session based, so only single-session behaviour was verified; multi-day stability and credential rotation over long runs were not exercised. Third, both cloud functions currently share one broadly scoped account role, as noted in the security discussion; splitting them into least-privilege roles is specified but was not possible in an account that cannot create new roles.

V. CONCLUSION

This work set out to move water treatment monitoring from periodic sampling toward continuous, compliance-aware observation, and to do so with a distributed design that places computation where it belongs: windowing, summarisation, and rule evaluation at a fog tier at the plant, durable storage and

elastic serving in the cloud. The result is a working pipeline that watches two plants and five streams each, flags turbidity, chlorination, acidity, and pressure breaches at the edge in the cycle they appear, and presents a live, compliance-annotated operator view drawn from a real cloud deployment.

Several lessons stand out. The value of the message queue was not latency but resilience: decoupling an imperfect plant uplink from the cloud consumer is what makes the pipeline tolerant of the conditions monitoring must survive. The sort-key design was a reminder that a small modelling decision, appending the plant identifier to a timestamp, is the difference between two plants coexisting and one silently overwriting the other. The most instructive lesson came from the deployment-only defect: a handler that passed every local test still failed on its first real invocation because the serverless runtime’s own calling convention collided with a test seam, which confirmed that live verification against the real account, not local testing alone, is what proves a serverless system works. Natural next steps are replicating the fog tier for high availability, splitting the shared role into least-privilege roles, and extending the

edge rules from fixed thresholds toward trend-based detection, so that a slow chlorine decline or a creeping turbidity rise is flagged from its trajectory rather than only when it crosses a line.

REFERENCES

- [1] World Health Organization, *Guidelines for Drinking-water Quality, 4th ed. incorporating the 1st addendum*. Geneva: World Health Organization, 2017.
- [2] N. Vijayakumar and R. Ramya, "The real time monitoring of water quality in IoT environment," in *Proc. Int. Conf. Innovations in Information, Embedded and Communication Systems (ICIIECS)*, 2015.
- [3] S. Geetha and S. Gouthami, "Internet of things enabled real time water quality monitoring system," *Smart Water*, vol. 3, no. 1, 2018.
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.
- [6] R. K. Naha, S. Garg, D. Georgakopoulos, P. P. Jayaraman, L. Gao, Y. Xiang, and R. Ranjan, "Fog computing: Survey of trends, architectures, requirements, and research directions," *IEEE Access*, vol. 6, pp. 47 980–48 009, 2018.
- [7] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011.
- [8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.
- [10] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. Sorenson, S. Sothikul, D. Terry, and A. Vig, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '22)*, 2022, pp. 1037–1048.
- [11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '18)*, 2018, pp. 133–146.
- [12] A. Barth, "The web origin concept," Internet Engineering Task Force, Tech. Rep. RFC 6454, 2011.